

## Домашнее задание №2

### Часть 1. Дополнение схемы

Новая таблица №1 — `lesson_progress` (прогресс прохождения уроков)

Бизнес-требование: «Платформа позволяет пользователям выбирать курсы и проходить обучение» — рекомендация из ДЗ №1: продумайте, как будете хранить статус завершения урока / курса.

До этого в модели фиксировался только статус курса целиком (`enrollments.status`), но не было видно, на каком конкретно уроке остановился студент. Таблица `lesson_progress` хранит по каждому уроку: пройден ли он, когда пройден и сколько секунд просмотрено.

Атрибуты:

- **enrollment\_id** (INTEGER, PK, FK → `enrollments`) — часть составного PK, запись студента на курс.
- **lesson\_id** (INTEGER, PK, FK → `lessons`) — часть составного PK, конкретный урок.
- **is\_completed** (BOOLEAN, NOT NULL, DEFAULT FALSE) — пройден ли урок.
- **completed\_at** (TIMESTAMP) — дата прохождения; пусто, пока урок не пройден.
- **watched\_seconds** (INTEGER, NOT NULL, DEFAULT 0, CHECK >= 0) — сколько секунд просмотрено.

Новая таблица №2 — `certificates` (сертификаты)

Бизнес-требование: студент, прошедший курс до конца, получает именной сертификат с уникальным номером, который можно проверить.

Связь с `enrollments` — 1:1: на одну запись на курс выдаётся не более одного сертификата. Реализуется через UNIQUE на внешнем ключе `enrollment_id`.

Атрибуты:

- **certificate\_id** (SERIAL, PK) — идентификатор сертификата.
- **enrollment\_id** (INTEGER, NOT NULL, UNIQUE, FK → `enrollments`) — запись на курс, за которую выдан сертификат.
- **certificate\_number** (VARCHAR(50), NOT NULL, UNIQUE) — публичный номер для проверки подлинности.
- **issued\_at** (TIMESTAMP, NOT NULL, DEFAULT NOW()) — дата выдачи.
- **final\_score** (INTEGER, CHECK BETWEEN 0 AND 100) — итоговый балл за курс.

Реализация связи M:N

В модели есть логическое отношение «многие ко многим»: один студент проходит много уроков, и один урок проходят много студентов.

В реляционной модели связь M:N нельзя выразить напрямую в таблице (нет типа «список ссылок»), а хранить перечень id через запятую в одной ячейке запрещает уже 1НФ (значения должны быть атомарны). Поэтому M:N разбивается на две связи 1:N через промежуточную связующую таблицу: enrollments (1) → lesson\_progress (N) ← (1) lessons.

Выбранный способ — связующая таблица с составным первичным ключом PRIMARY KEY (enrollment\_id, lesson\_id), как в таблице task\_tags из семинара.

Обоснование выбора:

- Составной РК сам по себе гарантирует, что одна и та же пара «запись на курс + урок» не появится дважды — отдельный UNIQUE не нужен.
- Суппрогатный ключ (progress\_id SERIAL) здесь не нужен: на эту таблицу никто не ссылается, а без UNIQUE(enrollment\_id, lesson\_id) он допускал бы дубликаты прогресса по одному уроку.
- Связующая таблица хранит не только сам факт связи, но и её атрибуты (is\_completed, completed\_at, watched\_seconds) — в этом её преимущество перед чистой связкой из двух столбцов.

Что будет при DROP TABLE ... CASCADE

CASCADE удаляет не только саму таблицу, но и все объекты, которые от неё зависят — в первую очередь внешние ключи других таблиц, ссылающиеся на удаляемую. Важно: удаляется ограничение, а не сама дочерняя таблица — строки в ней остаются, но теряют защиту целостности.

- **DROP TABLE lessons CASCADE;** — удалит таблицу уроков и заодно снесёт внешний ключ lesson\_progress.lesson\_id. Сама таблица lesson\_progress уцелеет, но её строки будут ссылаться в пустоту: проверять, что урок существует, станет нечем.
- **DROP TABLE enrollments CASCADE;** — удалит таблицу записей на курс и внешние ключи из lesson\_progress и certificates; сертификаты останутся «висеть» без владельца.
- **DROP TABLE lesson\_progress CASCADE;** — на эту таблицу никто не ссылается, поэтому CASCADE ничего дополнительного не удалит; эффект будет тот же, что у обычного DROP TABLE.

Отсюда вывод: в скрипте таблицы удаляются в порядке, обратном зависимостям (сначала дочерние, потом родительские). CASCADE оставлен как страховка на случай, если в базе остались объекты от прошлых прогонов — именно это делает скрипт воспроизводимым.

Актуализация индексов под частые запросы из ДЗ №1

- **idx\_lesson\_progress\_lesson\_id** — PostgreSQL автоматически индексирует составной РК целиком, то есть пару (enrollment\_id, lesson\_id). Такой индекс работает при поиске по первому столбцу, но не по второму. Поэтому запрос «сколько студентов прошли этот урок» без отдельного индекса шёл бы полным сканированием.

- **idx\_lesson\_progress\_completed** — частичный индекс (WHERE is\_completed = TRUE) под запрос №4 из ДЗ №1 («история обучения студента»). Индексируются только пройденные уроки, поэтому индекс заметно меньше обычного и быстрее читается.
- **idx\_certificates\_issued\_at** — под выборку выданных сертификатов за период (отчётность).
- Индекс на **certificates.enrollment\_id** отдельно не создавался: ограничение UNIQUE в PostgreSQL само создаёт уникальный индекс, и дублирующий индекс только замедлял бы вставку.

### Часть 3. Документирование нарушений

| № | Ограничение   | Выполняемый запрос (SQL)                                                                         | Сообщение СУБД                                                                                     | Понятное сообщение для пользователя                                                                                       | Как исправить                                                                                       |
|---|---------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1 | CHECK         | INSERT INTO reviews (...<br>rating)<br>VALUES (...<br>10)                                        | new row for relation "reviews"<br>violates check constraint<br>"reviews_rating_check"              | «Нельзя сохранить отзыв:<br>оценка курса<br>должна быть от 1<br>до 5 баллов»                                              | Указать оценку<br>в диапазоне от<br>1 до 5                                                          |
| 2 | FOREIGN KEY   | INSERT INTO lessons<br>(course_id, ...)<br>VALUES (999,<br>...)                                  | insert or update on table "lessons"<br>violates foreign key constraint<br>"lessons_course_id_fkey" | «Нельзя добавить<br>урок к курсу:<br>курс, к которому<br>его пытаются<br>привязать, не<br>существует»                     | Сначала<br>создать курс<br>либо указать<br>course_id<br>существующег<br>о курса                     |
| 3 | UNIQUE        | INSERT INTO enrollments<br>(user_id,<br>course_id, ...)<br>VALUES (3, 1,<br>...)                 | duplicate key value violates unique<br>constraint<br>"enrollments_user_id_course_id_key"<br>"      | «Нельзя записать<br>студента на курс:<br>он уже записан<br>на этот курс<br>ранее»                                         | Открыть<br>доступ к уже<br>существующей<br>записи вместо<br>создания новой                          |
| 4 | NOT NULL      | INSERT INTO users (name,<br>email, ...)<br>VALUES ('...',<br>NULL, ...)                          | null value in column "email" of<br>relation "users" violates not-null<br>constraint                | «Нельзя<br>зарегистрировать<br>пользователя без<br>email: почта<br>обязательна для<br>входа в систему»                    | Заполнить поле<br>email при<br>регистрации                                                          |
| 5 | Сложный CHECK | INSERT INTO lesson_progress (...<br>is_completed,<br>completed_at)<br>VALUES (...<br>TRUE, NULL) | new row for relation<br>"lesson_progress" violates check<br>constraint "lesson_progress_check"     | «Нельзя отметить<br>урок<br>пройденным без<br>даты<br>прохождения:<br>прогресс<br>студента стал бы<br>противоречивым<br>» | Указать<br>completed_at<br>вместе с<br>is_completed =<br>TRUE либо<br>оставить урок<br>непройденным |

Дополнительно продемонстрировано в скрипте

| Механизм           | Выполняемый запрос (SQL)                        | Результат                                                                                                      | Понятное сообщение для пользователя                                                             |
|--------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| ON DELETE CASCADE  | DELETE FROM enrollments WHERE enrollment_id = 1 | Вместе с записью удалились 3 строки прогресса и 1 сертификат                                                   | «При отмене записи на курс удаляются весь прогресс и выданный сертификат»                       |
| ON DELETE RESTRICT | DELETE FROM users WHERE user_id = 1             | update or delete on table "users" violates foreign key constraint "courses_teacher_id_fkey" on table "courses" | «Нельзя удалить преподавателя: у него есть созданные курсы, их история продаж была бы потеряна» |

### Краткие выводы

1. Ограничения целостности переносят бизнес-правила из кода приложения в саму базу данных. Правило «оценка от 1 до 5» соблюдается независимо от того, через какое приложение или скрипт пришли данные, — обойти его нельзя.
2. Техническое сообщение СУБД непригодно для конечного пользователя: строка `violates check constraint "reviews_rating_check"` ничего не говорит студенту. Поэтому в блоках `EXCEPTION` ошибка перехватывается и переводится на язык бизнеса, а `SQLERRM` сохраняется отдельно — для разработчика.
3. Перехват по конкретному типу ошибки (`check_violation`, `foreign_key_violation`, `unique_violation`, `not_null_violation`) точнее, чем универсальный `WHEN others`: он не замаскирует постороннюю ошибку в запросе под ожидаемое нарушение ограничения.
4. Выбор `ON DELETE` — это тоже бизнес-решение, а не техническая деталь. `CASCADE` (для прогресса и сертификата без записи на курс они бессмысленны) и `RESTRICT` (для преподавателя — его удаление уничтожило бы историю продаж) реализуют разную бизнес-логику одним и тем же механизмом внешнего ключа.
5. Связь `M:N` в реляционной модели всегда реализуется через связующую таблицу. Составной первичный ключ в ней одновременно решает две задачи — идентифицирует строку и запрещает дубликаты пары, — но требует отдельного индекса на второй столбец, так как индекс составного `PK` по нему не работает.